Prova Finale Reti Logiche

Politecnico di Milano

A.A. 2024/2025

Studente: Matteo Vignocchi

Matricola: 225614

Docente: Prof. Gianluca Palermo

Indice

1 Introduzione **2**
1.1 Elaborazione della specifica 2
1.2 Interfaccia componente 3
1.3 Esempio di funzionamento del filtro 3

2 Architettura **4**
2.1 Evoluzione dell'implementazione 4
2.2 FSA 4

3 Risultati **5**
3.1 Sintesi 5
3.2 Considerazioni temporali 6
3.3 Testbench 6

4 Conclusioni **7**

1 Introduzione

1.1 Elaborazione della specifica

Il progetto sviluppato per la prova finale del corso di *Reti Logiche* ha come obiettivo l'implementazione in VHDL di un modulo hardware dedicato al **filtraggio differenziale** di sequenze numeriche. Tale modulo è progettato per operare interfacciandosi con una memoria esterna (RAM), dalla quale legge in ingresso una sequenza di dati e sulla quale scrive i risultati dell'elaborazione. In particolare, lo sviluppo e la verifica sono stati effettuati utilizzando la **FPGA Xilinx XC7A200T-FBG484-1**, appartenente alla famiglia Artix-7.

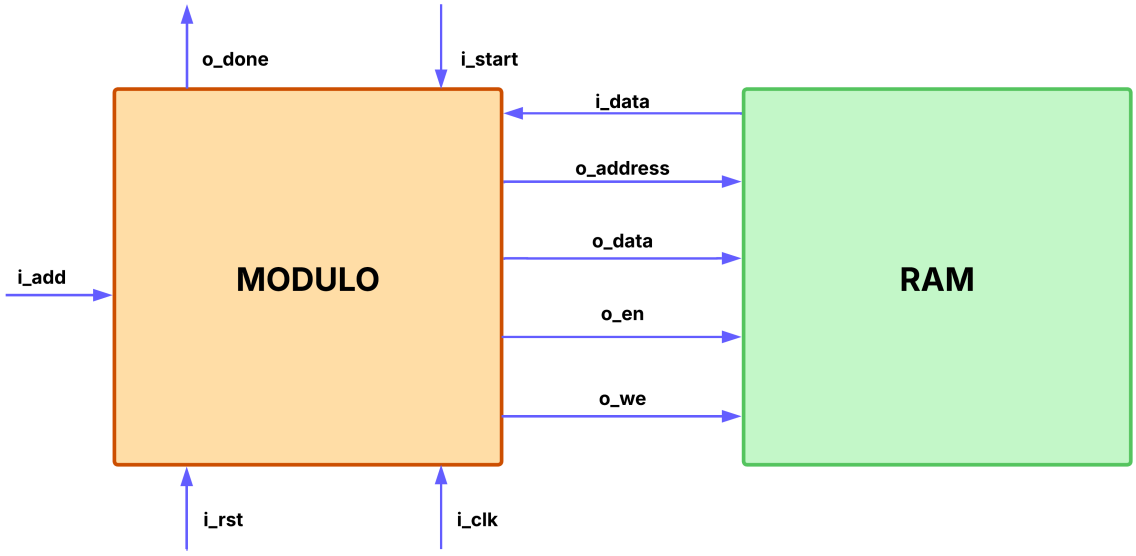

Figura 1. Diagramma ad alto livello del componente e della memoria

1.2 Interfaccia componente

L'interfaccia richiesta dal progetto è illustrata nella Figura 2.

```
entity project_reti_logiche is
    port (
        i_clk   : in std_logic;
        i_rst   : in std_logic;
        i_start : in std_logic;
        i_add   : in std_logic_vector(15 downto 0);

        o_done  : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

Figura 2. Schema dei principali segnali di ingresso e uscita del componente

In particolare, i vari segnali che compongono l'interfaccia sono descritti di seguito:

- *i_clk* — segnale di **clock** in ingresso, con periodo di 20 ns dato dal testbench;
- *i_rst* — segnale di **reset asincrono** che inizializza il modulo;
- *i_start* — segnale di avvio (**start**), mantiene il valore alto durante tutta l'elaborazione;
- *i_add* — indirizzo iniziale (**ADD**) in memoria dal quale parte la sequenza da elaborare;
- *o_done* — segnale di uscita che notifica la fine dell'elaborazione;
- *o_mem_addr* — indirizzo da inviare alla memoria esterna;
- *i_mem_data* — dato letto dalla memoria;
- *o_mem_data* — dato da scrivere in memoria;
- *o_mem_we* — segnale di **write enable** (1 per scrittura, 0 per lettura);
- *o_mem_en* — segnale di **enable** per l'accesso alla memoria.

1.3 Esempio di funzionamento del filtro

Per rendere più chiaro il comportamento del modulo, si riportano di seguito i coefficienti e un esempio numerico di elaborazione con il filtro differenziale di ordine 3.

Coefficienti del filtro di ordine 3

$$c = [\,0,\ -1,\ 8,\ 0,\ -8,\ 1,\ 0\,], \qquad n = 12$$

dove n rappresenta il fattore di normalizzazione.

Esempio numerico (valori in decimale)

- **Sequenza di partenza:** 32, -24, -35, 0, 46, -54, -39, -22, -53, -47, 12, 11, 11, 45, -30, -14, -35, -25, -19, -35, -41, -61, -24, -62
- **Dati filtrati con filtro di ordine 3 (prima della normalizzazione):** 157, 536, -178, -678, 428, 658, -355, 119, 251, -487, -400, 100, -314, 303, 426, ...
- **Sequenza finale dopo normalizzazione:** 11, 43, -13, -54, 33, 53, -28, 8, 18, -38, -31, 7, -24, 23, 33

2 Architettura

2.1 Evoluzione dell'implementazione

Fin dalle prime fasi di progettazione, l'approccio adottato è stato quello di modellare il sistema come una macchina a stati finiti (FSA). Tale scelta è stata guidata dall'analisi della specifica, la quale evidenzia una sequenza di azioni ripetitive che possono essere agevolmente descritte attraverso un ciclo di stati.

Inizialmente l'architettura era stata concepita con un numero ridotto di soli sette stati. Tuttavia, a seguito delle prime fasi di verifica e dei test non andati a buon fine, si è resa necessaria una revisione della struttura. L'implementazione è stata quindi estesa fino ai quattordici stati attuali.

Questa evoluzione è stata dettata dalla necessità di gestire in modo più accurato i segnali di ingresso e di uscita in corrispondenza del fronte di clock, garantendo una maggiore robustezza nelle operazioni di calcolo. Inoltre, in assenza di vincoli stringenti sulle prestazioni in termini di velocità di esecuzione, si è preferito adottare una soluzione più cauta, che privilegiasse la chiarezza del flusso di controllo e l'affidabilità del progetto.

2.2 FSA

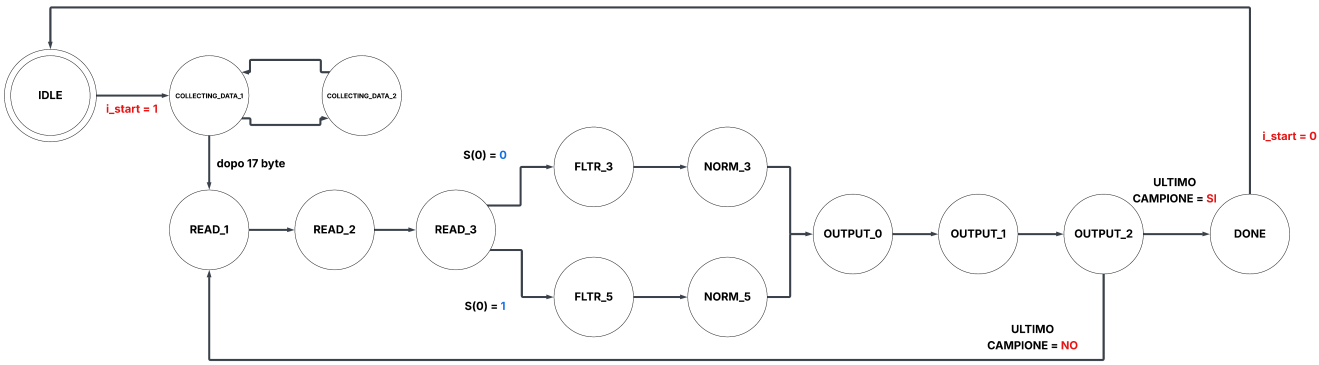

Figura 3. Macchina a stati finiti (FSA) del modulo project reti logiche

Qui riportata la spiegazione dei quattordici stati:

- **IDLE** — Stato iniziale e di attesa. Vengono azzerati registri e contatori. Al fronte attivo del segnale *i_start* vengono inizializzati indirizzi e variabili, e si passa all'acquisizione del preambolo.
- **COLLECTING_DATA_1** — Genera l'indirizzo di lettura per la memoria al fine di acquisire i 17 byte iniziali (lunghezza della sequenza, selettore del filtro e coefficienti).
- **COLLECTING_DATA_2** — Interpreta i dati letti: ricostruisce la lunghezza K, legge il selettore S (che determina l'ordine del filtro) e memorizza i coefficienti del filtro di ordine 3 e di ordine 5. Una volta completata la fase di raccolta, calcola l'indirizzo di uscita e passa alla lettura dei campioni.
- **READ_1**, **READ_2**, **READ_3** — Sequenza di stati che gestisce la lettura dei campioni da filtrare. Include la latenza della memoria e il caricamento dei dati in un registro scorrevole (finestra di 7 campioni), con inserimento di zeri di testa e di coda per la gestione dei bordi.
- **FLTR_3** — Calcola la convoluzione tra i campioni della finestra e i coefficienti del filtro di ordine 3.
- **FLTR_5** — Calcola la convoluzione utilizzando i coefficienti del filtro di ordine 5.
- **NORM_3** — Applica la normalizzazione del filtro di ordine 3, realizzata tramite approssimazione con shift a destra, con correzione per i valori negativi. Il risultato viene saturato nell'intervallo ammesso di 8 bit.
- **NORM_5** — Applica la normalizzazione del filtro di ordine 5 (approssimata con due shift), anch'essa con correzione per i valori negativi e saturazione a 8 bit.
- **OUTPUT_0**, **OUTPUT_1**, **OUTPUT_2** — Fase di scrittura del risultato: preparazione dell'indirizzo e del dato, attivazione dei segnali di *write enable* ed *enable*, e conclusione della scrittura. Se non tutti i campioni sono stati elaborati, si torna alla lettura del successivo.
- **DONE** — Stato finale di segnalazione. Porta alto *o_done* e rimane in attesa che *i_start* torni a zero, condizione che consente il ritorno a **IDLE**.

3 Risultati

3.1 Sintesi

La fase di sintesi del progetto è stata eseguita correttamente, confermando la coerenza strutturale e funzionale della descrizione in VHDL sviluppata. Il processo si è concluso senza errori e senza avvisi critici, restituendo il seguente esito:

Synthesis finished with 0 errors, 0 critical warnings and 14 warnings.

Il *report_utilization* ha evidenziato i seguenti dati relativi all'impiego delle risorse logiche della FPGA:

Tabella 1. Utilizzo delle risorse logiche post-sintesi

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	904	0	134600	0.67
LUT as Logic	904	0	134600	0.67
LUT as Memory	0	0	46200	0.00
Slice Registers	326	0	269200	0.12
Register as Flip Flop	326	0	269200	0.12
Register as Latch	0	0	269200	0.00
F7 Muxes	1	0	67300	<0.01
F8 Muxes	0	0	33650	0.00

Come si osserva, l'architettura non presenta alcun **latch**, confermando la corretta modellazione sincrona del progetto.

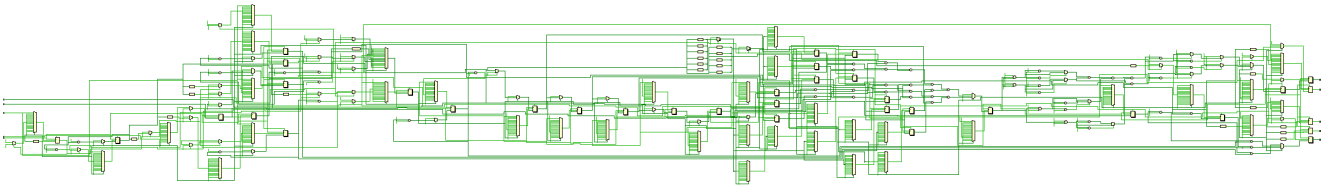

Figura 4. Schematico del componente

3.2 Considerazioni temporali

Per quanto riguarda l'analisi temporale, il *timing report* mostra un comportamento generalmente conforme ai requisiti, ma non privo di criticità. In particolare, pur rispettando i vincoli principali, la finestra temporale di 20 ns può risultare talvolta al limite a causa di possibili fenomeni di rumore e variazioni nell'ambiente operativo. In un'ottica più istituzionale, si può quindi affermare che, sebbene il progetto sia formalmente in linea con le specifiche, una sua applicazione in contesti particolarmente sensibili potrebbe beneficiare di ulteriori ottimizzazioni volte ad aumentare il margine di sicurezza temporale.

3.3 Testbench

Segue una sintesi dei principali test effettuati:

- **TB_Output_Isolation** — verifica che il DUT scriva 16 byte contigui a partire da `0x2121`, senza corrompere l'area di input, senza scritture dopo DONE e con handshake corretto.
- **TB_Address_Boundary** — controlla che, con indirizzi vicini al limite della memoria, le scritture arrivino fino a `0xFFFF` senza wrap-around né scritture fuori tempo.
- **TB_Consecutive_Runs** — esegue due elaborazioni consecutive con indirizzi diversi e senza reset intermedio, verificando che ciascun run rispetti il protocollo di scrittura e handshake.
- **TB_Saturation_Extremes** — genera valori di ingresso che portano a risultati saturati a ±128, controllando la corretta saturazione e il rispetto delle regole di output per entrambi i filtri.

- **TB_Coeff_Reload** — esegue due run con lo stesso ordine di filtro ma coefficienti diversi, verificando che il reload influenzi realmente l'output e che le scritture restino corrette.
- **TB_Filter3_BaseCase** — caso base con filtro d'ordine 3, verifica indirizzi, contiguità e corretto comportamento di START/DONE.
- **TB_Filter5_BaseCase** — caso base con filtro d'ordine 5, con stesse verifiche di correttezza temporale e spaziale delle scritture.
- **TB_Start_Glitch_Ignore** — inietta impulsi spuri di START durante l'elaborazione e verifica che vengano ignorati senza alterare le regole di scrittura.
- **TB_MaxK_Stress** — stress test con $K = 249$, filtro ordine 5, verifica contiguità, indirizzi e handshake senza overflow né wrap.
- **TB_MinK_Edges** — verifica i casi limite con sequenze piccolissime ($K = 1, 2, 3$) e conferma la correttezza delle scritture e dell'handshake.
- **TB_ZeroK** — caso $K = 0$, controlla che non avvenga alcuna scrittura, con handshake rispettato.
- **TB_MidRun_Reset** — forza un reset durante l'elaborazione, verificando assenza di scritture nel reset e ripartenza corretta alla successiva elaborazione.
- **TB_MinK_Allowed** — verifica il minimo $K = 7$, filtro ordine 3, con le stesse condizioni di validazione.
- **TB_SMask_Check** — dimostra che conta solo il bit meno significativo di S: run con $S = 0x00$ e $S = 0x82$ coincidono, con $S = 0x01$ differiscono.
- **TB_SetupReset** — reset durante il setup del preambolo, verifica assenza di scritture durante reset e ripartenza corretta.
- **TB_SignSymmetry** — verifica la simmetria di segno con filtro ordine 5: output con W e $-W$ risultano coerenti, salvo differenze di saturazione o ± 1 LSB.

Tutti i test hanno confermato la robustezza del progetto, evidenziando un comportamento coerente e ripetibile in scenari sia nominali sia di stress.

4 Conclusioni

Il componente sviluppato soddisfa pienamente le specifiche proposte nella consegna del progetto. L'implementazione è stata verificata sia in fase di simulazione pre-sintesi sia in ambiente post-sintesi, confermando la correttezza funzionale del modulo e la sua compatibilità con la piattaforma target.

Pur in presenza di alcuni *warnings* generati in fase di sintesi e di un margine temporale non ottimale (che in condizioni particolarmente rumorose potrebbe portare a situazioni limite), il progetto ha dimostrato una robustezza tale da superare senza difficoltà il *testbench* pubblico fornito dal corso.

In sintesi, il lavoro svolto ha permesso di:

- sviluppare un'architettura sincrona priva di *latch*;

- validare il corretto funzionamento attraverso un insieme ampio di testbench, inclusi casi limite e scenari di stress;
- garantire la conformità alle specifiche temporali e di I/O richieste.

La specifica del progetto non richiedeva limiti temporali e spaziali, il modulo quindi elaborato risulta complessivamente funzionale, anche se il risultato finale può essere migliorato dal punto di vista delle performance.